

# Programming Principles III

Week 03.



SOFTWARE CLUSTER

// COMT • **COMPUTATIONAL THINKING** (CIT1C18)



# Overview

- Learn how to create and use methods in Java;
- Learn how to create arrays in Java.



# Objectives

---

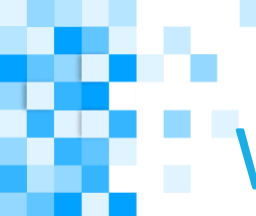
- Understand the concept of methods;
- Understand the concept of arrays;
- Demonstrate how to create and use methods in Java;
- Demonstrate how to create and use arrays in Java.



---

# METHODS

---



# What is a Method?

- A method (also known as function) is a **set of instructions** that are **grouped** together to form **an operation**.
- A method allows you to perform an operation more than one time by just writing the set of instructions once.

# Example

- Let's take the `TestRelational` program as an example.
- We can accomplish the same task by creating a method called `printRelationals()` and using it inside the `main()` method as shown in (B) below.

A

```
public class TestRelational
{
    public static void main(String args[])
    {
        int a = 10;
        int b = 20;

        System.out.println("a == b = " + (a == b) );
        System.out.println("a != b = " + (a != b) );
        System.out.println("a > b = " + (a > b) );
        System.out.println("a < b = " + (a < b) );
        System.out.println("b >= a = " + (b >= a) );
        System.out.println("b <= a = " + (b <= a) );
    }
}
```

B

```
public class TestRelational
{
    public static void printRelationals(int a, int b)
    {
        System.out.println("a == b = " + (a == b) );
        System.out.println("a != b = " + (a != b) );
        System.out.println("a > b = " + (a > b) );
        System.out.println("a < b = " + (a < b) );
        System.out.println("b >= a = " + (b >= a) );
        System.out.println("b <= a = " + (b <= a) );
    }

    public static void main(String args[])
    {
        printRelationals(10, 20);
    }
}
```



# Example

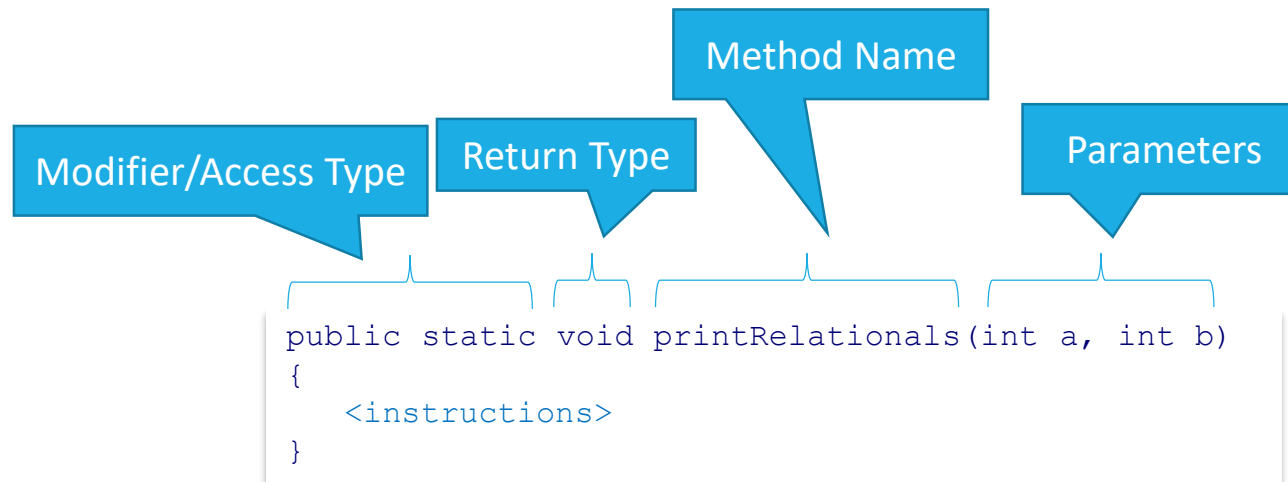
- We can also use the method again (as highlighted) and pass in a different set of values for variables **a** and **b** to give a different output.
- Modify the **TestRelational** program as shown on the right.
- Compile and run the program to see the output.
- Notice that without creating and using a method, we will have to type the whole set of codes again in order to evaluate different values of **a** and **b**.

```
public class TestRelational
{
    public static void printRelationals(int a, int b)
    {
        System.out.println("When a = " + a + ", b = " + b);
        System.out.println("a == b = " + (a == b) );
        System.out.println("a != b = " + (a != b) );
        System.out.println("a > b = " + (a > b) );
        System.out.println("a < b = " + (a < b) );
        System.out.println("b >= a = " + (b >= a) );
        System.out.println("b <= a = " + (b <= a) );
    }

    public static void main(String args[])
    {
        printRelationals(10, 20);
        System.out.println("=====");
        printRelationals(45, 28);
    }
}
```

# Structure of a Method

- A method is written in the form as shown below:



- Modifier/Access Type** – this defines how the method can be accessed. It is optional in certain cases such as when creating object classes.
- Return Type** – if a method provides a result from the operation, the data type of the result must be specified. Otherwise, `void` is used.
- Method Name** – The method name given should be meaningful. It must start with lower case letter.
- Parameters** – these are similar to variables. A parameter defines the dynamic values that you need to perform the operation. Each parameter is created by specifying a data type and a variable name. Parameters are optional, a method may contain no parameters if they are not required. Multiple parameters are added using the comma as shown above.





# The Return Type

- The `printRelationals` example method does not provide a result from the operation. Notice that what the method does is just printing out values and results to the console.
- What if you want a method to provide a result from the operation?
- Let's take a look at another example on the next slide.

# The Return Type

- In this example, a program called `TestRectangle` is created and one of its operations is to calculate the area of a rectangle when the length and breadth are provided.
- The method `calculateArea()` is created to do so.
- Create this `TestRectangle` program using Notepad++ and run it to see the result.
- Proceed to the next slide.

```
1 public class TestRectangle
2 {
3     public static int calculateArea(int length, int breadth)
4     {
5         System.out.print("Calculating area of rectangle with ");
6         System.out.println("length = " + length + " cm and breadth = " + breadth + " cm");
7
8         int area = length * breadth;
9
10        return area;
11    }
12
13    public static void main(String args[])
14    {
15        int areaOfRectangle = calculateArea(10, 30);
16        System.out.println("Area of rectangle = " + areaOfRectangle + " cm sq");
17    }
18 }
```

# The Return Type

- Let's take a closer look at the method `calculateArea()`.

We use `int` as the return data type here because the result is contained in the variable called `area` which we created using the `int` data type. You can use other data types depending on the data type of the result that the operation provides.

```
public static int calculateArea(int length, int breadth)
{
    System.out.print("Calculating area of rectangle with ");
    System.out.println("length = " + length + " cm and breadth = " + breadth + " cm");

    int area = length * breadth;

    return area;
}
```

This instruction with the `return` keyword tells the method to provide the result, which is contained in the variable called `area`. **Note that no other instructions should be created after the `return` line.**

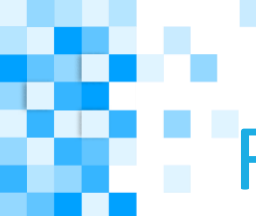
# Calling Method

- Let's take a closer look at how the method `calculateArea()` is used in the `main()` method.
- To use a method, we **call** its name and provide the values of the parameters if required.

When a method provides the result of an operation, it "holds on" to the result. The result can be assigned to a variable as shown here.

```
public static void main(String args[])
{
    int areaOfRectangle = calculateArea(10, 30);
    System.out.println("Area of rectangle = " + areaOfRectangle + " cm sq");
}
```

- A method can be called in any other methods, not necessary only in the `main()` method.



# Fun Quiz

- A method is written as shown below. How should this method be called or used?

```
public static void methodOne()  
{  
    System.out.println("Hello");  
}
```

- a) methodOne;
- b) methodOne();
- c) MethodOne();
- d) methodOne("World");



# Fun Quiz

- What is wrong with the following method?

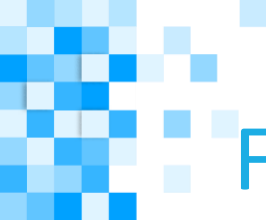
```
public static void methodTwo()  
{  
    int result = 10 + 32;  
  
    return result;  
}
```



# Fun Quiz

- What is wrong with the following method?

```
public static String methodThree(String message)
{
    String msg = message + ". Add another sentence.";
}
```



# Fun Quiz

- What is wrong with the following method?

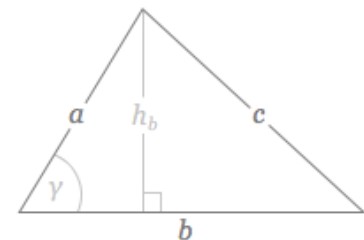
```
public static String methodFour(name)
{
    String message = "Happy Birthday " + name + "!!!";

    return message;
}
```



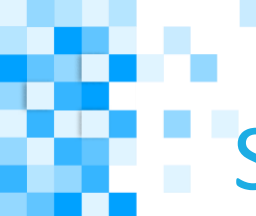
# Challenge

1. In the `TestRectangle` program, suppose you want to provide more precision to the result such that the method `calculateArea()` can calculate the area of a rectangle with length and breadth that have decimal places. Modify the method `calculateArea()` so that it provides the result with decimal places.
2. In the same program, create a new method that tests whether a rectangle is a square and provide a result of true or false.
3. Create a new program that contains three methods (You need to create the three methods):
  - i. Calculate the perimeter of a triangle;
  - ii. Calculate the area of a triangle.
  - iii. Tests if a triangle is equilateral, and provide a result of true or false.





# VARIABLE SCOPE

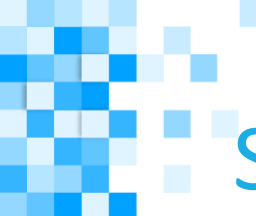


# Scope of Variables

- Can two variables have the same name in the same program?
- The answer is Yes and No. This is because it depends on where the variable is.
- For example, the variables `age` in the codes on the right will produce an error because both have the same name in the same method.
- The system would be confused as to which `age` to use.

```
public class TestProgram
{
    public static void main(String args[])
    {
        int age = 17;
        int age = 9;

        System.out.println("John is " + age + " years old.");
    }
}
```



# Scope of Variables

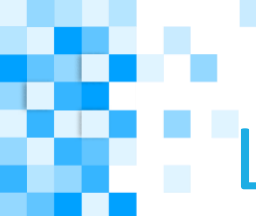
- However, if the same variable name is used in different methods, then it is allowed because they belong to different methods, as shown on the right.
- This is known as the scope of variables.
- There are basically two scopes of variables:
  - Local
  - Class

```
public class TestProgram
{
    public static void printJohnAge()
    {
        int age = 17;

        System.out.println("John is " + age + " years old.");
    }

    public static void main(String args[])
    {
        int age = 9;

        System.out.println("Lisa is " + age + " years old.");
        printJohnAge();
    }
}
```



# Local Variables

- **Local variables** can only be used within its confinement.
- The **age** variables in the example on the right are local variables because they can only be used in the methods that they are created in.

```
public class TestProgram
{
    public static void printJohnAge()
    {
        int age = 17;

        System.out.println("John is " + age + " years old.");
    }

    public static void main(String args[])
    {
        int age = 9;

        System.out.println("Lisa is " + age + " years old.");
        printJohnAge();
    }
}
```



# Local Variables

- The variable `x` in the example on the right can only be used within the `for` loop because it is created in it.

```
public class TestProgram
{
    public static void main(String args[])
    {
        int y = 3;
        <instructions>

        for (int i = 0; i < 10; i++)
        {
            int x = 8;
            <instructions>

            y++;
        }

        x = x + 2; // wrong because x cannot be found
    }
}
```



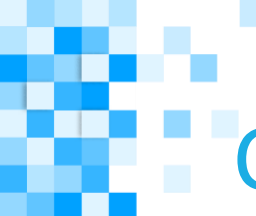
# Local Variables

- The variable `x` in the example on the right can only be used within the `if` statement because it is created in it.

```
public class TestProgram
{
    public static void main(String args[])
    {
        int i = 10;
        int y = 3;
        <instructions>

        if (i < 10)
        {
            int x = 8;
            <instructions>
            y++;
        }

        x = x + 2; // wrong because x cannot be found
    }
}
```



# Class Variables

- A class variable can be used anywhere within the class.
- In the example on the right, variables `name1` and `name2` can be used anywhere in the class.
- They are created outside of all methods but within the class.

```
public class TestProgram
{
    public static String name1 = "John";
    public static String name2 = "Lisa";

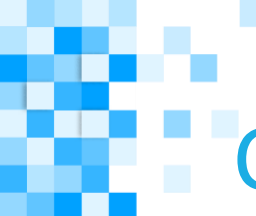
    public static void printJohnAge()
    {
        int age = 17;

        System.out.println(name1 + " is " + age + " years old.");
    }

    public static void main(String args[])
    {
        int age = 9;

        System.out.println(name2 + " is " + age + " years old.");
        printJohnAge();
    }
}
```





# Constant

- The variable called `DEPARTMENT` is known as a **constant**.
- A **constant** is a variable whose value can never be changed after it is created.
- The name of a constant is always in upper case letters.
- A constant is created by using **public static final** followed by the data type and variable name as shown on the right.

```
public class TestProgram
{
    // DEPARTMENT is a constant
    public static final String DEPARTMENT = "Games";

    public static String name1 = "John";
    public static String name2 = "Lisa";

    public static void printJohnAge()
    {
        int age = 17;

        System.out.println(name1 + " is " + age + " years old.");
    }

    public static void main(String args[])
    {
        int age = 9;

        System.out.println(name2 + " is " + age + " years old.");
        printJohnAge();

        System.out.println(name2 + " works in the " + DEPARTMENT + " department.");
    }
}
```



---

---

# ARRAY



# What is an Array?

- You have been taught about variables so far. A variable can only store **one** value at a time.
  - What if you want to store a list of values using only one variable?
  - For example, you may want to store:
    - a list of friends' names; or
    - a list of months from January to December; or
    - a list of months using digits 1 to 12 instead.
  - You can do so in coding using **Array**.
  - So how do you write an **Array**?
  - There are two major steps to creating an Array as shown on the right.
1. Declare or create the Array variable with a specific size.
  2. Store the values or items one-by-one into the Array.

# How to Create an Array?

Assuming you want to store a list of 5 friends' names.

This part is the data type of information that you want to store in the Array.

In this case we use `String` since we are storing names.

This is how you store each name into the Array one-by-one.

The number in the "`[]`" here refers to the index of the item that you store in the array.

The index of an Array always starts from `zero`.

So for 5 items, the index ends at 4.

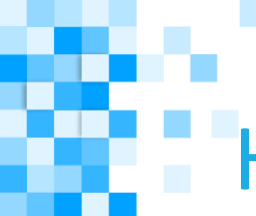
An Array is represented by "`[]`".

This is the variable name for the array. We use `friends` since it is more meaningful here.

```
// 1. Declare or create the Array variable with a specific size
String[] friends = new String[5];

// 2. Store the values or items one-by-one into the Array
friends[0] = "Jane Chong";
friends[1] = "Tommy Lee";
friends[2] = "Varun Dada";
friends[3] = "Syafique";
friends[4] = "Taylor Swift";
```

This part tells the system to create a new Array of 5 strings.



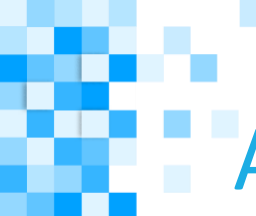
# How to Create an Array?

- In Java, an array can be created in any of the following ways:

1) `String[] friends = new String[5];`

2) `String []friends = new String[5];`

3) `String friends[] = new String[5];`



# Adding/Storing Items in Array

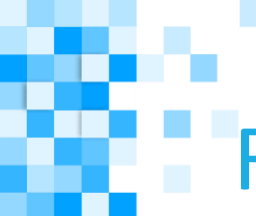
- You can add items to an array in two ways:

```
String[] friends = new String[5];  
  
friends[0] = "Jane Chong";  
friends[1] = "Tommy Lee";  
friends[2] = "Varun Dada";  
friends[3] = "Syafique";  
friends[4] = "Taylor Swift";
```

**OR**

```
String friends[] = {"Jane Chong", "Tommy Lee", "Varun Dada", "Syafique", "Taylor Swift"};
```

- An array item always starts from index 0.
- All items in an array must belong to the same data type which the array variable is created as.



# Retrieving Items from Array

- To retrieve items from an array, we just call the array name with the square bracket and indicate the index of the item we want to retrieve in the bracket.
- For example, the following retrieve item 3 in the `friends` array and assign to a variable called `myFriend`:

```
String myFriend = friends[2];
```

- What will the value of `myFriend` be after executing the above statement?

# Try It Out!

- The following is a simple program to print the items in an array.
- We can use a **for** loop to go through the entire array.
- Use your Notepad++ or other editor to create this program.
- Compile and run to see the result.

```
1  public class TestArray
2  {
3      public static void main(String args[])
4      {
5          int numbers[] = new int[5];
6
7          numbers[0] = 10;
8          numbers[1] = 20;
9          numbers[2] = 70;
10         numbers[3] = 40;
11         numbers[4] = 50;
12
13         // The following 'for' loop prints the items in the array
14         // 'length' is the property of array that
15         // gives the size or number of items in it.
16         for (int index = 0; index < numbers.length; index++)
17         {
18             System.out.println(numbers[index]);
19         }
20     }
21 }
```

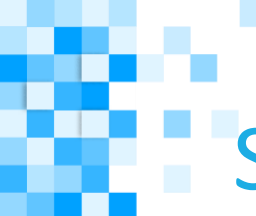




# Summary

---

- In this lesson, you have learned the concepts of methods, arrays, and the scope of variables.
- In the next lesson, you will start to learn the basic concept of object and classes in object-oriented programming.



# Supplementary Materials

| Topic        | Source                                                                                                                                                                                                                                               |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Methods | <a href="https://www.tutorialspoint.com/java/java_methods.htm">https://www.tutorialspoint.com/java/java_methods.htm</a>                                                                                                                              |
| Java Static  | <a href="http://www.baeldung.com/java-static">http://www.baeldung.com/java-static</a><br><a href="http://blog.jbaysolutions.com/2016/08/16/understanding-static-in-java/">http://blog.jbaysolutions.com/2016/08/16/understanding-static-in-java/</a> |

